

PRD: B-Roll Director

LLM-as-Director architecture for AI-assisted footage discovery

Author: Erfan Salehi | Version: 0.2 (Architecture rewrite) | Date: May 2026

1. Overview

1.1 What changed from v0.1

v0.1 used a fragmented pipeline: split the script into ~315 timestamp slots, ask a chat LLM (Groq) to generate keywords for each slot in batches of 5-6, then search YouTube. The LLM never saw the whole script at once, never knew what shot it suggested 6 seconds earlier, and treated every slot as equally important.

v0.2 inverts the model. The LLM acts as a video director: it reads the entire script in one pass, produces a structured shot list with intent, shot type, search queries, and priority per timestamp, then optionally curates the search results returned from stock footage and YouTube APIs. The LLM is provider-abstracted — Gemini Flash, Groq Llama 3.3, DeepSeek, OpenRouter, or local Ollama can all play the role.

1.2 Why this is better

- **Whole-script context.** The LLM sees narrative flow and can avoid visual repetition (no 'newsroom shot' four times in eight seconds).
- **Fewer API calls.** One large call replaces ~50 batched calls. Easier on rate limits, faster wall-clock time.
- **Richer output per slot.** Shot intent, shot type, duration needed, priority — not just keywords.
- **Provider-agnostic.** If Groq's free tier changes or Gemini removes free Flash access, swap the provider via config. No code changes downstream.
- **Optional curation pass.** After search APIs return candidates, the LLM can rank them based on the shot intent it originally specified — closing the loop.

1.3 Goals

- Produce a structured shot list for a 10-minute video in under 90 seconds (single LLM call).
- Keep total time-to-research under 10 minutes including stock and YouTube lookups.
- Run on free-tier services for 2-3 videos/week without hitting paid usage.
- Allow swapping LLM providers via a single config setting.
- Deliver output files an editor can open in any NLE (txt, srt, optional XML/EDL) without installing plugins.

1.4 Non-goals (v1)

- Video-frame-level analysis of clips (visual validation by a multimodal model). Reserved for v1.5.
- Auto-downloading clips. Reserved for v1.5.

- Plugin-based NLE integration (Premiere/Resolve).
- Multi-language support (English script only).
- Cloud deployment or multi-user accounts.

2. Architecture

2.1 Three stages

Stage 1 — Direct: LLM reads full script + voiceover duration. Returns structured shot list as JSON.

Stage 2 — Search: For each shot in the list, query stock footage APIs (Pexels, Pixabay) and YouTube (yt-dlp) using the search queries the LLM generated. Pure metadata fetching — no LLM calls.

Stage 3 — Curate (optional): Send the candidate metadata back to the LLM in a single batched call. LLM ranks the top 3 candidates per shot based on the shot intent it originally specified. Outputs final ranked shot list.

2.2 Why three stages instead of one

The naive approach is 'just send everything to the LLM' — but that conflates two different problems. Stage 1 is a creative task (interpret the script). Stage 3 is a ranking task (pick the best from a list). Splitting them lets us:

- Pause between stages so the editor can review and tweak the shot list before burning search-API quota.
- Cache stage 1 output by script hash. Re-runs cost nothing if the script hasn't changed.
- Skip stage 3 entirely on simple videos where stage 2's first result is usually good.
- Use different models per stage if useful (e.g. cheap Flash-Lite for Stage 3 ranking, better model for Stage 1 directing).

3. LLM Provider Abstraction

3.1 The interface

All LLM calls go through a thin internal interface. The application code never imports a vendor SDK directly. A single config file picks the active provider:

```
# config.yaml llm:   provider: gemini   # or: groq, openrouter, deepseek, ollama
               model:  gemini-3-flash-preview   api_key_env: GEMINI_API_KEY
max_tokens: 16000  temperature: 0.4
```

Adding a new provider = adding one Python file (~50 lines) implementing two methods: `complete(prompt)` and `complete_json(prompt, schema)`. Everything else is shared.

3.2 Supported providers (v1)

Provider	Recommended Model	Free Tier	Context	Notes
Google AI	Gemini 3 Flash	1,500 RPD,	1M tokens	Best free option for

Provider	Recommended Model	Free Tier	Context	Notes
Studio		1M ctx		Stage 1. Whole-script fits comfortably.
Groq	llama-3.3-70b-versatile	1,000 RPD, 6K TPM	128K	Fastest inference (~300 tok/sec). TPM cap may bite on long scripts.
OpenRouter	:free routes (Llama, Gemma, etc.)	Varies by model	Varies	Aggregator. Useful for fallback. Quality varies, models change.
DeepSeek	DeepSeek V4 Flash	Paid, ~\$0.18/M	128K	No free tier but cheapest paid option. Good fallback for high volume.
Ollama (local)	qwen2.5:14b or llama3.1:8b	Free (local)	Model-dependent	Offline fallback. Quality below cloud models. Needs ~16GB RAM.

3.3 Default and fallback chain

If the configured provider returns an error or hits a rate limit, the tool tries the next in the fallback chain rather than failing the run:

1. Primary: Gemini 3 Flash (largest free quota, best whole-script handling)
2. Secondary: Groq Llama 3.3 70B (fast, good fallback)
3. Tertiary: OpenRouter free route (last-resort)

Fallback is configurable. A user with no Gemini key can set Groq as primary.

4. Inputs & Outputs

4.1 Inputs

Input	Required	Notes
Script (.txt)	Yes	Source of truth for content. Must match voiceover word-for-word.
Voiceover (.mp3, .wav, .m4a)	Yes	Used for total duration only. No transcription. Timing inferred proportionally.
LLM API key	Yes	For the active provider. Saved to local .env via UI. Never logged.
Pexels API key	Optional	Free, instant signup. Without it, only YouTube results are returned.
Pixabay API key	Optional	Free, manual approval. Adds variety to stock results.

Input	Required	Notes
Style hints	Optional	Free-text field, e.g. 'cinematic, slow motion, no talking heads, no shorts'. Passed to the director prompt verbatim.

4.2 Output files

4.2.1 shot_list.json — the canonical output

Machine-readable structured shot list. Source of truth for everything else.

```
[ { "slot_id": 1, "timestamp_start": "00:00:00",
  "timestamp_end": "00:00:01", "script_chunk": "Welcome back to the
channel", "shot_intent": "establish energy, channel branding",
"shot_type": "wide", "search_queries": ["youtube intro animation",
"studio welcome shot"], "duration_needed_sec": 1.5, "priority":
"high", "candidates": [ { "source": "pexels", "url": "...",
"title": "...", "duration": 12, "ranked": 1 }, { "source": "youtube",
"url": "...", "title": "...", "ranked": 2 } ] }, ... ]
```

4.2.2 shot_list.txt — human-readable summary

Plain text version of the shot list, easier to read for editors who don't want JSON.

```
[00:00:00 - 00:00:01] PRIORITY: high SHOT: wide Script: Welcome back to
the channel Intent: establish energy, channel branding Queries: youtube
intro animation | studio welcome shot Top picks: 1. [Pexels] Studio Welcome
Shot (12s) https://... 2. [YouTube] Channel Intro Templates 2026
https://... 3. [Pixabay] Camera Recording Studio https://...
```

4.2.3 timing.srt — NLE-importable guide track

Standard SRT. Each subtitle shows top pick title + script chunk, so editors see both when scrubbing the timeline:

```
1 00:00:00,000 --> 00:00:01,000 "Welcome back to the channel" → Studio
Welcome Shot [Pexels] 2 00:00:01,000 --> 00:00:02,000 "Today we're talking
about" → Presenter Pointing at Screen [Pexels]
```

4.2.4 download_list.txt — flat URL list

One URL per line, top pick per slot only. Drop into a download manager to grab everything in one go.

4.2.5 markers.fcpxml (optional)

Final Cut Pro XML containing one timeline marker per slot, named with the shot intent and top keyword. Importable into Premiere Pro and DaVinci Resolve as 'Import XML'. Editor sees all markers on the timeline with descriptions, no plugin install required.

5. The Director Prompt (Stage 1)

5.1 Inputs to the prompt

- Full script text

- Voiceover duration in seconds
- Intro duration setting (default 30s) and pacing rules (1 shot/sec for intro, 1 shot/2sec for body)
- Optional style hints from the user

5.2 What the LLM is asked to do

4. Estimate timing: words/second based on script length and audio duration.
5. Generate a shot list covering every required slot. Compute timestamps.
6. For each slot, produce: shot intent, shot type, 2-3 visual concept search queries, duration needed, priority (high/medium/low).
7. Avoid visual repetition across adjacent slots. The model has the whole script — it should use it.
8. Mark slots that should stay on the talking head (no B-roll) with priority='none' and empty queries — based on phrases like 'as I mentioned earlier' or direct address moments.
9. Return strict JSON conforming to the schema. No prose, no markdown.

5.3 Prompt design principles

- **Visual concepts, not literal keywords.** Bad: "economy 2008". Good: "stock market crash trading floor panic", "newspaper headlines recession 2008".
- **Shot type vocabulary.** Constrain the LLM to a fixed set: wide, medium, close-up, extreme close-up, aerial, abstract, graphic. Keeps output usable downstream.
- **Priority is meaningful.** 'high' = key emotional or narrative moments where the right shot really matters. 'low' = filler. Used in Stage 3 to decide where to spend curation tokens.
- **Few-shot examples in the prompt.** Three worked examples (a typical educational sentence, an emotional beat, a transition phrase) anchor output quality.

5.4 Token budget

- Input: ~500 token system prompt + ~2,000 token script (10-min video) + ~200 token style hints = ~2,700 tokens.
- Output: ~315 slots × ~40 tokens per slot = ~12,000-14,000 tokens.
- Total: ~16,500 tokens per video. Fits in every provider's context. Free on Gemini, ~\$0.04 on paid Gemini Flash.

6. The Curator Prompt (Stage 3, optional)

6.1 What it does

After Stage 2 returns a list of candidate clips per slot (with title, tags, duration from Pexels/Pixabay/yt-dlp), the curator pass sends batches of slots back to the LLM to rank candidates. The LLM compares each candidate's metadata to the shot intent it generated in Stage 1 and picks the best 3.

6.2 What it does NOT do

It does not watch the video. No frame analysis. No video tokens. Pure text-on-text reasoning over metadata. This is the difference between Stage 3 (cheap, free-tier-friendly) and v1.5's video validation (paid, multimodal).

6.3 Cost

- Input per batch: $10 \text{ slots} \times (\text{shot intent} + 10 \text{ candidates} \times 50 \text{ tokens metadata}) = \sim 6,000$ tokens.
- Output per batch: $10 \text{ slots} \times \text{ranking} = \sim 500$ tokens.
- Total per video: $\sim 32 \text{ batches} \times 6,500 \text{ tokens} = \sim 210\text{K}$ tokens.
- Free on Gemini Flash. $\sim \$0.30$ on paid Gemini Flash. Skippable entirely if results from stage 2 look good.

7. User Interface (Streamlit)

7.1 Single-page flow

10. **Setup (collapsible).** LLM provider dropdown. API key inputs for active provider, Pexels, Pixabay. 'Test connection' button per provider.
11. **Upload.** Script (.txt) and voiceover (.mp3/.wav/.m4a). Show detected duration and word count.
12. **Settings.** Intro duration (default 30s). Style hints (free text). Shot pacing override.
13. **Direct (Stage 1).** 'Generate shot list' button. Single LLM call, $\sim 30\text{-}90$ sec. Progress shown. Results displayed as an editable table — every field can be tweaked inline.
14. **Search (Stage 2).** 'Find footage' button. Parallel calls to Pexels, Pixabay, yt-dlp. Progress bar.
15. **Curate (Stage 3, optional).** 'Rank with AI' button. Greyed out if user wants to skip.
16. **Download.** Buttons for shot_list.json, shot_list.txt, timing.srt, download_list.txt, markers.fcpxml. Plus 'Download all (.zip)'.

7.2 Design principles

- Stages are explicit and pause-able. The editor controls when to spend API quota.
- Every output is editable before download.
- Provider/key changes don't require restarting the app.
- Persistent local cache keyed by script hash. Re-running the same video is free and instant.

8. Technical Stack

Component	Choice
Language	Python 3.10+
UI	Streamlit (single-file app, runs at http://localhost:8501)
LLM abstraction	Custom thin layer + LiteLLM library for OpenAI-compatible

Component	Choice
	providers (Groq, OpenRouter, DeepSeek share schema). Native SDK only for Gemini.
Structured output	Pydantic models for shot list schema. JSON Schema mode where supported (Gemini, OpenAI-compatible); fallback regex-extract for providers without it.
Audio metadata	mutagen for duration extraction (no ffmpeg dependency)
Stock APIs	Pexels REST API (httpx async client), Pixabay REST API
YouTube search	yt-dlp with ytsearch3: prefix; --match-filter for duration bounds (30s-30min)
Concurrency	asyncio for stock APIs, ThreadPoolExecutor for yt-dlp (which is blocking)
Caching	diskcache library, keyed by SHA256(script + settings + provider + model)
Distribution	Single repo with run.bat / run.sh launcher: creates venv, installs deps, opens browser

8.1 Project structure

```

broll-director/ ├── app.py                                # Streamlit UI |── core/ |
|── timing.py                                # Voiceover duration + slot generation |── |
director.py    # Stage 1: prompt + JSON parsing |── |── search/ |
|── pexels.py |── |── pixabay.py |── |── youtube.py |── #
yt-dlp wrapper |── curator.py                    # Stage 3: ranking pass |
|── output.py    # txt, srt, json, fcpxml writers |── llm/ |
|── base.py      # Abstract Provider interface |── |── gemini.py
|── groq.py |── |── openrouter.py |── |── deepseek.py |── |── ollama.py
|── prompts/ |── |── director.md                    # Stage 1 system prompt |── |
curator.md    # Stage 3 system prompt |── schemas/ |── |
shot_list.py  # Pydantic models |── config.yaml |── |
Default config (provider, models) |── requirements.txt |── run.sh / run.bat
|── README.md

```

9. Cost & Quota Analysis

9.1 Per video, with Gemini 3 Flash (free tier)

- Stage 1: 1 call, ~16K tokens. Free.
- Stage 2: ~315 stock API calls + ~315 yt-dlp searches. Free (Pexels, Pixabay, yt-dlp).
- Stage 3 (optional): ~32 calls, ~210K tokens. Free.
- **Total: free.** Comfortably within free-tier limits for 2-3 videos/week.

9.2 Per video, with Groq Llama 3.3 70B (free tier)

- Stage 1: 1 call, ~16K tokens — but Groq's TPM cap is 6,000/min. Single call exceeds this.
- **Mitigation:** Split Stage 1 into 3-4 sub-calls (script chunked into 2-3 minute segments) when using Groq. Each call ~5K tokens, fits in TPM budget. Costs an extra ~30 seconds total wall time.
- Stage 3: ~32 calls, ~210K tokens spread across the run. Fine on TPM.
- **Total: free, ~9-12 minutes wall-clock per video.**

9.3 Per video, paid (DeepSeek V4 Flash, ~\$0.18/M)

- Stage 1: ~16K tokens × \$0.18/M = ~\$0.003
- Stage 3: ~210K tokens × \$0.18/M = ~\$0.04
- **Total: ~\$0.05 per video.** If free tiers ever break, paid fallback is trivially cheap.

10. Risks & Open Questions

10.1 Risks

Risk	Likelihood / Impact	Mitigation
LLM returns malformed JSON	Medium / High — known issue with non-flagship models	Use JSON mode where available; fallback to regex-extract + repair pass; retry up to 2x with stricter prompt
Free-tier quotas tighten (Google did this in Dec 2025 and April 2026)	High / Medium	Provider abstraction is the entire point — config swap, no code change. Document fallback chain in README.
Groq TPM cap on a single Stage 1 call	Certain / Low	Provider-specific chunking strategy in groq.py. Other providers use single call.
Whole-script context still produces visual repetition	Low / Medium	Iterate on the director prompt with explicit anti-repetition rules; track in pilot
yt-dlp breaks on YouTube changes	Medium / High — happens 2-3x/year	Pin known-good version; pip install -U yt-dlp on launch; document manual update path
Stock API rate limits (Pexels 200/hr)	Low / Low	315 calls/video over ~5 minutes is well under the limit; throttle if multiple videos run consecutively

10.2 Open questions

- Does Stage 3 actually improve results enough to justify the extra time, or is Pexels' default ranking already good enough? Test on 5 real videos before deciding to default it on or off.
- Should the director prompt receive the previous video's shot list (if any) to maintain channel-level visual consistency? Useful for series content.
- Should Stage 1 detect script structure (intro/main/outro/CTA) automatically and apply different pacing per section, instead of the user-defined `intro_duration`?
- Should we expose multiple alternative shot lists ('show me 3 different directors' takes')? Cheap to do — one extra API call. Useful or overwhelming?

11. Milestones

Week	Milestone	Deliverable
1	LLM abstraction + Stage 1	Provider interface + Gemini and Groq implementations. Director prompt v1. CLI: <code>input script.txt + voice.mp3 -> output shot_list.json</code> . No UI.
2	Stage 2 search	Pexels + Pixabay + yt-dlp clients. Output <code>shot_list.txt</code> and <code>shot_list.json</code> with candidates populated.
3	Streamlit UI v0	File uploaders, provider selector, editable shot list table, progress bars, download buttons.
4	Stage 3 + outputs	Curator prompt + ranking integration. SRT and FCPXML writers. Bootstrap launcher. README.
5	Pilot + iteration	Run on 3-5 real videos. Tune director prompt. Add OpenRouter and Ollama providers if needed.

12. Success Metrics

- **Time-to-shot-list (Stage 1 only):** under 90 seconds for a 10-min video on Gemini Flash, under 3 minutes on Groq.
- **Time-to-full-research:** under 10 minutes for a 10-min video including Stage 2.
- **Shot-intent accuracy:** editor rates >70% of shot intents as 'a real director would have written this' on first 5 videos.
- **Top-pick usability:** editor uses the #1-ranked candidate as-is for >50% of slots, without picking #2 or #3.
- **Visual repetition:** manually count repeated shots within 8-second windows; target <5% of slots.
- **Provider portability:** can swap from Gemini to Groq via config and produce comparable output, no code change.
- **Adoption:** at least 1 editor running it independently within 2 weeks of pilot.

13. v1.5 and Beyond

13.1 v1.5 — Visual validation

Add a multimodal pass for high-priority slots. The LLM (Gemini Flash, OpenAI 4o-mini, or Claude) watches the top candidate clip and confirms it matches the shot intent, returning a best-segment timestamp (e.g. 'the relevant shot is at 02:14-02:22'). Combined with yt-dlp's --download-sections flag, this enables auto-trim downloads — only the relevant 8 seconds, not the full clip.

- Cost: ~\$0.30-0.50 per video on paid Gemini Flash, applied only to ~30 high-priority slots.
- Saves the editor from scrubbing 50 downloaded clips looking for the right moment.

13.2 v2.0 — Auto-download and pre-trim

- Tool downloads top picks directly into a project folder using Pexels/Pixabay direct URLs and yt-dlp --download-sections.
- Folder structure: project_name/00_00_02_intro.mp4, 00_00_03_presenter.mp4, etc.
- Editor drags the folder into Premiere/Resolve and clips are pre-named, pre-trimmed, ready to drop on the timeline.

13.3 v2.5+ — Optional NLE integration

- Once the Premiere MCP ecosystem stabilizes, add a one-button 'Send to Premiere' that places clips on the timeline at correct timestamps.
- Treat as opt-in — the FCPXML output covers 80% of the workflow benefit without plugin install.

13.4 Other deferred ideas

- Multi-language scripts (Persian first, given Tovo Studio context).
- Channel-level visual consistency (pass previous shot lists into the director prompt).
- Alternative shot lists ('show me 3 directors' takes').
- Editor feedback loop — track which top picks the editor actually uses and feed back into prompt tuning.